

MHM — MoneyHub Mobile

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.1

Campo	Descrição
Projeto	MHM — MoneyHub Mobile (cliente Flutter do MoneyHub)
Gerente de Projeto	Leonardo Lamin
Gerente/Responsável de GCS-DevOps	Leonardo Lamin
Fábrica de Software / Squad	LL — Leonardo Lamin (desenvolvimento individual)
Repositório principal	https://github.com/LaminLeonardo/moneyhub-mobile

Histórico de Revisões

Data	Versão	Descrição	Autor
10/08/2026	1.0	Elaboração do documento (template) com enfoque em práticas DevOps (CI/CD).	Luiz Camargo
22/09/2026	1.1	Adaptação do template ao projeto MoneyHub Mobile: escopo, papéis, ferramentas, ambientes, pipelines de CI/CD, IaC, controle de mudanças, baselines, métricas e marcos.	Leonardo Lamin

Sumário

1. Introdução.....	3
1.1 Finalidade.....	3
1.2 Escopo.....	3
1.3 Definições, Acrônimos e Abreviações.....	3
1.4 Referências.....	4
1.5 Visão Geral.....	4
2. Cultura DevOps e Papéis na Gerência de Configuração.....	4
3. Ferramentas, Ambientes e Infraestrutura.....	5
3.1 Ferramentas do Pipeline.....	5
3.2 Ambientes e Infraestrutura.....	7
3.3 Estratégia de Branching e Versionamento.....	7
4. Identificação da Configuração.....	8
4.1 Itens de Configuração.....	8
4.2 Convenção de Nomenclatura.....	8
4.3 Estrutura do Repositório.....	9
5. Integração Contínua (CI).....	9
5.1 Etapas do Pipeline de CI.....	9
5.2 Política de Pull Request e Revisão de Código.....	10
5.3 Critérios de Qualidade (Quality Gate).....	10
6. Entrega e Implantação Contínua (CD).....	10
6.1 Etapas do Pipeline de CD.....	10
6.2 Estratégias de Implantação.....	11
6.3 Rollback e Recuperação.....	11
7. Infraestrutura como Código (IaC).....	12
7.1 Versionamento de Infraestrutura.....	12
7.2 Ambientes Efêmeros.....	12
8. Controle de Configuração e Mudança.....	12
8.1 Processo de Solicitação de Mudança.....	12
8.2 Comitê de Controle de Mudança (CCM).....	13
8.3 Correções Emergenciais (Hotfix).....	13
9. Baselines e Releases.....	13
9.1 Geração Automática de Baseline.....	13
9.2 Notas de Versão (Release Notes).....	14
10. Estimativa do Status de Configuração.....	14
10.1 Monitoramento e Observabilidade.....	14
10.2 Métricas DORA.....	14
10.3 Relatórios e Auditorias.....	15
11. Marcos.....	15
12. Treinamento e Recursos.....	15
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros.....	16
14. Quadro-Resumo do Pipeline CI/CD.....	16

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/indivíduo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGCS é destinado ao desenvolvedor individual **Leonardo Lamin**, que exerce todos os papéis descritos na seção 2, responsável pelo desenvolvimento e operação do **MoneyHub Mobile** — cliente Flutter (Android e Web) do MoneyHub, aplicativo de finanças pessoais que permite acompanhar saldo, receitas, despesas e lançamentos por categoria. O plano abrange todos os itens de configuração do repositório **moneyhub-mobile** (código, testes, pipelines, Dockerfile, IaC e documentação) e os ambientes de Desenvolvimento, Homologação e Produção. O backend .NET e o frontend React do MoneyHub possuem repositórios próprios e ficam fora do escopo; a futura integração com a API é tratada como dependência externa (seção 13).

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
IC	Item de Configuração — artefato sob controle de versão e de mudanças
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças (Change Control Board — CCB)
SM	Solicitação de Mudança (Change Request), registrada como Issue no GitHub
PR	Pull Request — pedido de integração de uma branch na main, com revisão e pipeline
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Build	Versão executável gerada automaticamente pelo pipeline a partir de um commit
Release	Versão identificada (tag vX.Y.Z) e empacotada para entrega; estabelece nova baseline
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.

Termo	Significado
Conventional Commits	Padrão de mensagens de commit (feat, fix, docs, chore...) que permite versionamento e changelog automáticos
SAST / SCA	Análise estática de segurança do código / Análise de composição (dependências) de software
GHCR	GitHub Container Registry — registro de imagens Docker (ghcr.io)
Smoke test	Teste rápido pós-deploy que verifica se a aplicação está no ar e na versão esperada
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate) + Rework Rate

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.
- Aulas de Gerência de Configuração de Software (GCS-PS), Aulas 02 a 06 — Prof. Luiz Carlos Camargo, 2026 (check-in/check-out, build, release, tags, branch/merge, controle de mudanças, CCB, planejamento de GCS).
- Repositório do projeto: <https://github.com/LaminLeonardo/moneyhub-mobile> (README, CONTRIBUTING, docs/onboarding.md, docs/processo-mudanca.md).
- ADRs do projeto: docs/adr/0001-flutter-como-cliente-mobile.md e docs/adr/0002-trunk-based-e-release-please.md.
- Conventional Commits 1.0.0 — conventionalcommits.org; Semantic Versioning 2.0.0 — semver.org.
- Documentação oficial do GitHub Actions, GitHub Pages, GitHub Environments e GitHub Container Registry.
- DORA — Accelerate State of DevOps Report (métricas de entrega de software).

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps** é **responsabilidade compartilhada entre desenvolvimento e operações** ("**Você constrói, você executa**" — **you build it, you run it**), **apoiada por automação**. Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Prioriza o backlog, autoriza mudanças de maior impacto e acompanha indicadores de entrega (DORA Metrics).
Engenheiro DevOps / Release Engineer	Projeta e mantém os pipelines de CI/CD, define a estratégia de <i>branching</i> e versionamento, administra o registro de artefatos/imagens e a IaC.

Papel	Responsabilidades
Desenvolvedor(a)	Integra código com frequência na branch principal, escreve testes automatizados, revisa Pull Requests e é corresponsável pela qualidade do que é implantado.
SRE / Operações <i>Site Reliability Engineering</i>	Monitora ambientes de produção, define SLIs/SLOs, participa de postmortems e realiza rollback quando necessário. - <i>Service Level Indicators (SLIs) and Service Level Objectives (SLOs)</i> .
QA / Automação de Testes	Mantém a suíte de testes automatizados (unitários, integração, aceitação) executada nos pipelines de CI e CD.
Segurança (DevSecOps)	Define políticas de análise estática (SAST), análise de dependências (SCA) e gestão de segredos integradas ao pipeline.
Comitê de Controle de Mudanças (CCM)	Atua apenas em mudanças de alto risco/impacto (ex.: mudanças em produção fora do fluxo padrão); mudanças de rotina seguem o fluxo automatizado de PR + pipeline.

Por se tratar de um projeto individual, os papéis acima são acumulados pelo responsável do projeto, conforme a atribuição abaixo. A revisão de código por pares ("quatro olhos") é garantida por um colega convidado como colaborador do repositório.

Papel	Responsável no MoneyHub Mobile
Gerente de Projeto / Product Owner	Leonardo Lamin — prioriza o backlog (GitHub Issues/Projects) e acompanha as métricas DORA
Engenheiro DevOps / Release Engineer	Leonardo Lamin — mantém ci.yml, cd.yml, rollback.yml, Dockerfile, release-please e infra/terraform
Desenvolvedor(a)	Leonardo Lamin — implementa as SMs em branches de feature e escreve os testes
Revisor(a) de Pull Request	Colega da turma convidado como colaborador (permissão Write) — aprova PRs
SRE / Operações	Leonardo Lamin — acompanha deploys, smoke tests e executa rollback
QA / Automação de Testes	Leonardo Lamin — mantém testes unitários, de widget e de integração (flutter test)
Segurança (DevSecOps)	Leonardo Lamin, apoiado por gitleaks, Dependency Review e Dependabot no pipeline
Comitê de Controle de Mudanças (CCM)	Leonardo Lamin (responsável GCS) + professor da disciplina como stakeholder consultado (seção 8.2)

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas efetivamente adotadas pelo projeto (todas gratuitas para repositório público):

Categoria	Ferramenta(s) adotada(s)	Finalidade
Controle de versão	Git + GitHub (repositório público moneyhub-mobile)	Versionamento de código-fonte, testes, IaC, pipelines e documentação

Categoria	Ferramenta(s) adotada(s)	Finalidade
Orquestração de CI/CD	GitHub Actions (ci.yml, cd.yml, rollback.yml, iac.yml, labels.yml)	Automação de build, testes, análise, empacotamento, implantação e rollback
Build / SDK	Flutter SDK (canal stable) e Dart 3	Geração do build Web e do APK Android a partir do mesmo commit
Empacotamento	Docker (imagem nginx:1.27-alpine)	Imagem imutável e versionada do build Web
Registro de artefatos	GitHub Container Registry (ghcr.io), GitHub Releases e Actions Artifacts	Imagens Docker, APK e pacote Web (.zip) de cada release
Hospedagem dos ambientes	GitHub Pages (branch gh-pages)	Homologação (/staging) e Produção (/)
Infraestrutura como Código	Terraform (provider integrations/github) + HCP Terraform (estado remoto)	Repositório, proteção da main e environments versionados
Gestão de segredos	GitHub Secrets e GitHub Environments	GITHUB_TOKEN efêmero e RELEASE_PLEASE_TOKEN; nenhum segredo no código
Qualidade de código e testes	dart format, flutter analyze (flutter_lints), flutter test, lcov	Padronização, análise estática, testes e cobertura mínima
Segurança / SCA	gitleaks, GitHub Dependency Review, Dependabot	Deteção de segredos e de vulnerabilidades em dependências
Versionamento e release	release-please + Conventional Commits	Cálculo da versão SemVer, CHANGELOG, tag e GitHub Release
Monitoramento e Observabilidade	Histórico do GitHub Actions, aba Deployments, smoke tests, notificações de falha	Estado dos pipelines e dos ambientes
Rastreamento de tarefas	GitHub Issues (templates SM e Defeito), labels e GitHub Projects	Vinculação de SMs a branches, commits, PRs e releases
Comunicação	GitHub Discussions (categoria Announcements)	Canal único de avisos do projeto
Ambiente de desenvolvimento	VS Code (extensões Flutter e Dart), Git Bash, Android Studio (emulador)	Edição, execução e depuração local

Equipamentos e política de atualização: o desenvolvimento ocorre no notebook pessoal do desenvolvedor (Windows); os pipelines executam em runners hospedados do GitHub (ubuntu-latest), efêmeros e recriados a cada execução. O Flutter SDK segue o canal stable e é atualizado no início de cada iteração; dependências (pub, GitHub Actions, imagem Docker e provider Terraform) são atualizadas por PRs automáticos do Dependabot, que passam pelo mesmo pipeline de CI.

3.2 Ambientes e Infraestrutura

Os ambientes devem ser, sempre que possível, provisionados via Infraestrutura como Código, garantindo paridade entre eles e permitindo recriação rápida e rastreável.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Máquina local (flutter run) e execução do CI em runner efêmero a cada PR; o build web do PR fica disponível como artefato (web-build-ci, 7 dias).	Abertura/atualização de Pull Request
Homologação / Staging	https://laminleonardo.github.io/moneyhub-mobile/staging/ — mesmo build que irá para produção; validação funcional e smoke test automático.	Merge na main (CD automático, environment homologacao)
Produção	https://laminleonardo.github.io/moneyhub-mobile/ + APK anexado à GitHub Release + imagem ghcr.io/laminleonardo/moneyhub-mobile:X.Y.Z.	Criação da release (merge do PR de release) + aprovação manual no environment producao

Detalhamento: DNS e HTTPS são gerenciados pelo GitHub Pages (domínio laminleonardo.github.io); não há servidores, IPs ou banco de dados próprios — o app é servido como site estático por CDN e, nesta fase, mantém os dados em memória. Credenciais: o GITHUB_TOKEN é gerado por execução e o RELEASE_PLEASE_TOKEN fica em GitHub Secrets. A imagem Docker permite executar qualquer versão localmente: `docker run -p 8080:80 ghcr.io/laminleonardo/moneyhub-mobile:latest`.

Política de uso do ambiente:

- Nenhum commit direto na main; toda mudança chega via PR aprovado e com CI verde.
- A branch gh-pages e os ambientes são alterados exclusivamente pelos pipelines, nunca manualmente.
- Homologação é sobrescrita a cada merge na main; Produção só recebe versões com tag de release.
- Segredos, chaves de assinatura e arquivos .env nunca são versionados (bloqueados pelo .gitignore e pelo gitleaks).

3.3 Estratégia de Branching e Versionamento

Estratégia adotada: **Trunk-Based Development (GitHub Flow)**, com branches de vida curta:

- **main** — sempre implantável; protegida por PR obrigatório com 1 aprovação, status checks obrigatórios (pr-title, build-test, security, docker-build), histórico linear, sem force push e sem bypass para administradores.
- **feature/<issue>-<descrição-curta>** (e **fix/<issue>-...**) — criada a partir da main atualizada, com duração máxima de 2 dias; excluída automaticamente após o merge.
- **release/x.y** — não utilizada: a estabilização da versão é feita pelo PR de release automático do release-please (chore: release x.y.z).
- **hotfix/<issue>-<descrição>** — correções emergenciais criadas a partir da main (que corresponde à produção) e integradas pelo mesmo fluxo de PR; se necessário, executa-se antes o rollback (6.3).
- **gh-pages** — branch técnica mantida exclusivamente pelo pipeline de CD (conteúdo publicado nos ambientes).

Versionamento: **Versionamento Semântico** (MAJOR.MINOR.PATCH) e **Conventional Commits**. O merge é sempre por squash, e o título do PR (validado no CI) torna-se o commit na main. O release-please lê esses commits, calcula a próxima versão, atualiza pubspec.yaml e CHANGELOG.md e cria a tag vX.Y.Z. Enquanto MAJOR = 0, feat incrementa MINOR e fix incrementa PATCH.

4. Identificação da Configuração

4.1 Itens de Configuração

Itens de configuração do MoneyHub Mobile, todos versionados no repositório moneyhub-mobile:

- Código-fonte da aplicação (mobile/lib) e testes automatizados (mobile/test: unit, widget e integration)
- Pipelines como código: .github/workflows/ci.yml, cd.yml, rollback.yml, iac.yml e labels.yml
- Dockerfile, nginx.conf e .dockerignore (mobile/)
- Infraestrutura como Código: infra/terraform/*.tf
- Configuração por ambiente: base href aplicado no deploy, APP_VERSION via --dart-define e environments homologacao/producao — segredos ficam somente em GitHub Secrets
- Scripts de migração de banco de dados: N/A (o cliente não possui banco de dados próprio)
- Documentação técnica: README, CONTRIBUTING, CHANGELOG, docs/onboarding.md, docs/processo-mudanca.md, ADRs (docs/adr) e este PGCS (docs/pgcs)
- Dependências travadas: pubspec.yaml + pubspec.lock; configuração de qualidade (analysis_options.yaml)
- Configuração de GCS: release-please-config.json, .release-please-manifest.json, .github/dependabot.yml, .github/labels.yml, CODEOWNERS e templates de Issue/PR

Não são ICs (artefatos derivados, reproduzíveis pelo pipeline e ignorados no .gitignore): build/, coverage/, .dart_tool/, arquivos .tfstate e .env, chaves de assinatura.

Ciclo de vida dos artefatos:

- **Draft** — artefato em uma branch de feature: mudanças livres e rápidas, apenas sob controle de versão.
- **Aceito** — após PR revisado, aprovado e com CI verde, integrado à main: torna-se IC e só muda via nova SM.
- **Manutenção** — após a release em produção: alterações somente por SM (melhoria) ou defeito/hotfix.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<n° issue>-<descrição-curta>	feature/42-filtro-categoria
Branch de correção	fix/<n°>-<descrição> hotfix/<n°>-<descrição>	hotfix/60-tela-branca
Commit / título de PR	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(lancamentos): adiciona filtro por categoria
Issue	[SM] <título> [DEFEITO] <título>	[SM] Filtro por categoria

Item	Convenção	Exemplo
Tag de release	Versionamento Semântico vX.Y.Z	v0.2.0
Imagem de container	ghcr.io/laminleonardo/moneyhub-mobile:<versão sha-<commit> main latest>	ghcr.io/laminleonardo/moneyhub-mobile:0.2.0
Artefatos da release	moneyhub-<tag>.apk moneyhub-web-<tag>.zip	moneyhub-v0.2.0.apk
Versão exibida no app	<versão>+<sha curto do commit>	0.2.0+a1b2c3d
Arquivos Dart / testes	snake_case.dart <nome>_test.dart	finance_summary_test.dart
ADR	docs/adr/NNNN-<título>.md	0002-trunk-based-e-release-please.md

4.3 Estrutura do Repositório

Repositório único (<https://github.com/LaminLeonardo/moneyhub-mobile>), organizado da seguinte forma:

Caminho	Conteúdo
.github/workflows/	Pipelines como código: ci.yml, cd.yml, rollback.yml, iac.yml, labels.yml
.github/ISSUE_TEMPLATE/	Formulários de Solicitação de Mudança (SM) e de Relato de Defeito
.github/	pull_request_template.md, labels.yml, dependabot.yml, CODEOWNERS
mobile/lib/	Código-fonte Flutter: models, services, screens, config
mobile/test/	Testes unitários (unit/), de widget (widget_test.dart) e de integração (integration/)
mobile/ (raiz)	pubspec.yaml/.lock, analysis_options.yaml, Dockerfile, nginx.conf, CHANGELOG.md, android/, web/
infra/terraform/	IaC da plataforma GitHub (repositório, proteção da main, environments)
scripts/	check_coverage.sh (quality gate de cobertura) e smoke_test.sh (verificação pós-deploy)
docs/	onboarding.md, processo-mudanca.md, adr/ e pgcs/ (este plano em .docx e .pdf)
raiz	README.md, CONTRIBUTING.md, LICENSE, .gitignore, release-please-config.json, .release-please-manifest.json

5. Integração Contínua (CI)

Todo Pull Request para a branch principal deve disparar automaticamente um pipeline de CI. Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

5.1 Etapas do Pipeline de CI

1. Checkout do código e restauração de dependências com cache (actions/checkout, subosito/flutter-action no canal stable, flutter pub get com versões travadas pelo pubspec.lock).
2. Build da aplicação: flutter build web --release (o APK é gerado no pipeline de CD).
3. Análise estática e padrões: dart format --set-exit-if-changed, flutter analyze (flutter_lints) e validação do título do PR em Conventional Commits (job pr-title).

4. Testes unitários e de widget com flutter test --coverage e verificação de cobertura mínima (scripts/check_coverage.sh).
5. Testes de integração (mobile/test/integration): fluxo completo de cadastro de lançamentos — telas, repositório e cálculo de saldo.
6. Segurança: gitleaks (segredos no histórico), Dependency Review (SCA das dependências alteradas no PR) e analisador do Dart como SAST (o Dart não é suportado pelo CodeQL); alertas do Dependabot ativos.
7. Empacotamento: job docker-build gera a imagem nginx com o build web; em PR a imagem é apenas construída — a publicação no GHCR ocorre no CD.

Os jobs **pr-title**, **build-test**, **security** e **docker-build** do workflow ci.yml são status checks obrigatórios na proteção da main.

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige 1 aprovação do revisor (colega colaborador) antes do merge ("quatro olhos"); novos commits invalidam aprovações anteriores.
- PRs pequenos (até ~300 linhas) e de vida curta (máx. 2 dias); janela de resposta a PRs de até 2 dias úteis, com avisos no GitHub Discussions.
- Merge somente com pipeline de CI verde (quality gate), exclusivamente por squash merge.
- Todo PR referencia sua SM (Closes #<n>) e preenche o checklist do template de PR.

5.3 Critérios de Qualidade (Quality Gate)

O PR só pode ser integrado se atender a todos os critérios abaixo:

- Formatação: 0 arquivos fora do padrão dart format.
- Análise estática: 0 erros e 0 warnings no flutter analyze (unused_import, unused_local_variable e dead_code tratados como erro).
- Testes: 100% dos testes automatizados passando.
- Cobertura de linhas $\geq 70\%$ (lcov), com meta de 80% até a versão 1.0.0.
- Segurança: 0 segredos detectados (gitleaks) e 0 dependências novas com vulnerabilidade alta/crítica (Dependency Review).
- Título do PR em Conventional Commits; build web e imagem Docker gerados com sucesso.

6. Entrega e Implantação Contínua (CD)

Após a publicação do artefato pelo pipeline de CI, o pipeline de CD é responsável por promover essa mesma versão, sem recompilação, através dos ambientes até a produção.

No MoneyHub Mobile, a cada merge na main o workflow cd.yml gera **uma única vez** o build web, o APK e a imagem Docker (job build). Os jobs seguintes apenas promovem esse mesmo artefato, alterando somente a configuração do ambiente (base href).

6.1 Etapas do Pipeline de CD

8. Deploy automático em Homologação (job deploy-homologacao): publicação do build web em gh-pages:/staging.

9. Verificação automatizada em Homologação (job smoke-homologacao): o site responde, carrega main.dart.js e expõe a versão esperada em version.json; a validação funcional é registrada na SM (status:verificada). Testes E2E automatizados estão planejados (marco M9).
10. Gate de aprovação para produção: o release-please mantém o PR de release; ao ser aprovado e integrado, cria a tag vX.Y.Z e a GitHub Release, e o job deploy-producao aguarda aprovação manual no environment producao.
11. Deploy em produção com substituição atômica do site (ver 6.2) e promoção da imagem Docker (mesmo digest) para as tags X.Y.Z e latest.
12. Testes de fumaça (smoke tests) pós-deploy em produção com scripts/smoke_test.sh.
13. Baseline/tag de release e notas de versão: tag vX.Y.Z, CHANGELOG.md e GitHub Release com o APK e o pacote web anexados.

6.2 Estratégias de Implantação

Estratégia	Descrição
Rolling update	Substituição gradual das instâncias antigas pelas novas, minimizando indisponibilidade.
Blue-Green	Duas versões (ambientes) coexistem; o tráfego é comutado de uma vez para a nova versão, permitindo rollback imediato.
Canary Release	A nova versão recebe uma pequena fração do tráfego real, sendo ampliada gradualmente conforme a confiança.
Feature Flags	Funcionalidades são implantadas desligadas e ativadas em produção de forma independente do deploy, reduzindo o risco de releases.

Estratégia adotada: substituição atômica com retorno imediato (variação simplificada de Blue-Green) — cada publicação no GitHub Pages troca todos os arquivos de uma vez por um novo commit na gh-pages, e a versão anterior permanece disponível (artefatos da release anterior e imagem no GHCR). O ambiente de Homologação valida previamente o mesmo build. Rolling update e Canary não se aplicam a hospedagem estática; Feature Flags via --dart-define serão usadas para funcionalidades incompletas (ex.: integração com a API).

6.3 Rollback e Recuperação

Toda implantação deve ser reversível. O pipeline de CD deve manter a versão anterior disponível (imagem no registro, IaC versionada) para permitir rollback automatizado em caso de falha nos testes de fumaça ou nos indicadores de saúde pós-deploy.

No projeto, o workflow **rollback.yml** (acionamento manual informando a tag, ex.: v0.1.0) baixa o pacote moneyhub-web-<tag>.zip da GitHub Release, republica-o em produção sem recompilar, reaponta a imagem latest para a versão restaurada e executa o smoke test.

- Acionamento: falha no smoke test de produção ou defeito de severidade crítica.
- Meta de restauração (MTTR): menos de 1 hora.
- Retenção: releases, tags e imagens publicadas não são apagadas, garantindo que qualquer baseline possa ser restaurada.

7. Infraestrutura como Código (IaC)

Toda a infraestrutura provisionada para os ambientes de Desenvolvimento, Homologação e Produção deve ser descrita como código versionado, submetida ao mesmo fluxo de Pull Request e pipeline usado para a aplicação.

No MoneyHub Mobile, a infraestrutura é a própria plataforma de entrega no GitHub. Ela é descrita em `infra/terraform` com o provider `integrations/github`: `github_repository` (squash merge, discussions, alertas de vulnerabilidade), `github_branch_protection` (regras da main) e `github_repository_environment` (homologacao e producao, esta com revisor obrigatório). As labels do processo de mudança também são código (`.github/labels.yml`, sincronizadas pelo workflow `labels.yml`).

7.1 Versionamento de Infraestrutura

- Alterações em `infra/**` seguem PR + revisão + pipeline: o workflow `iac.yml` executa `terraform fmt -check` e `terraform validate`; o `plan/apply` é executado pelo responsável de GCS após a aprovação.
- O estado da infraestrutura é armazenado de forma centralizada no HCP Terraform (workspace `moneyhub-mobile-github`), nunca em máquinas locais; o token de acesso é fornecido por variável de ambiente.

7.2 Ambientes Efêmeros

Os runners de CI são efêmeros (recriados a cada execução). Review apps por PR não serão adotados nesta fase, pois o GitHub Pages hospeda um único site por repositório; como alternativa, o build web de cada PR é publicado como artefato do pipeline (`web-build-ci`) para validação isolada.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão: abertura de item de trabalho → branch de feature → Pull Request → revisão → pipeline de CI → merge → pipeline de CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança.

A Solicitação de Mudança (SM) é registrada com o template de Issue "Solicitação de Mudança", que contém: identificação única (nº da issue), solicitante, IC(s) afetado(s), classificação (nova funcionalidade, melhoria, correção, infraestrutura), prioridade, descrição, análise de impacto, critérios de aceitação e indicação de necessidade do CCM. Defeitos usam o template "Relato de Defeito" (versão, ambiente, severidade, passos, resultado esperado x obtido). A situação é controlada por labels:

Situação (label)	Significado
<code>status:nova</code>	SM registrada, aguardando análise
<code>status:atribuida</code>	Analisada, priorizada e com responsável
<code>status:em-andamento</code>	Branch de feature/fix criada
<code>status:finalizada</code>	PR integrado na main com CI verde (issue fechada por Closes #nº)
<code>status:verificada</code>	Validada no ambiente de Homologação

Situação (label)	Significado
status:fechada	Entregue em Produção (consta no CHANGELOG da release)
status:rejeitada	Não será implementada — justificativa registrada na issue

8.2 Comitê de Controle de Mudança (CCM)

O CCM é acionado apenas em mudanças de alto impacto ou risco elevado, como alterações estruturais de infraestrutura, mudanças de esquema de banco de dados em produção, ou exceções ao fluxo padrão (ex.: hotfix emergencial). Em situações rotineiras, a aprovação por pares (revisão de PR) substitui o comitê formal.

Composição: Leonardo Lamin (responsável GCS/PO) e o professor da disciplina como stakeholder consultado. As decisões são registradas como comentário na própria SM, com as labels ccm:aprovada ou ccm:rejeitada. **O CCM é acionado (label ccm:necessario) quando a mudança:**

- altera a proteção da main, os environments ou qualquer arquivo em infra/;
- altera os pipelines de CD ou de rollback (cd.yml, rollback.yml);
- é um hotfix aplicado fora do fluxo padrão;
- é uma breaking change (feat!) ou altera o contrato com a API do MoneyHub;
- amplia o escopo acordado da iteração ou tem custo maior que o benefício percebido.

8.3 Correções Emergenciais (Hotfix)

- Mesmo em emergência, cria-se a issue de defeito (severidade Crítica, label hotfix) para garantir rastreabilidade.
- Se a produção estiver indisponível, executa-se primeiro o workflow Rollback com a última tag estável.
- A correção segue em branch hotfix/<nº>-<descrição>, com PR, revisão e CI verde — o pipeline nunca é pulado.
- Após a entrega, o CCM decide se a correção é mantida ou refatorada (com novos casos de teste).

9. Baselines e Releases

9.1 Geração Automática de Baseline

Diferente de modelos tradicionais, em que a baseline é criada manualmente em marcos do projeto, neste plano cada implantação bem-sucedida em produção gera automaticamente uma baseline: uma tag de versão (SemVer) associada ao commit, à imagem publicada no registro e ao estado da infraestrutura correspondente.

Baselines definidas para o projeto:

- **Baseline inicial v0.1.0** — estrutura do repositório, app inicial, pipelines e PGCS (primeira GitHub Release, criada manualmente).
- **Baselines de produto vX.Y.Z** — geradas automaticamente pelo release-please a cada release, vinculando commit, tag, imagem no GHCR, APK e pacote web.
- **Baseline de documentação** — o PGCS versionado em docs/pgcs, com o histórico de revisões atualizado a cada alteração.

9.2 Notas de Versão (Release Notes)

As notas de release são geradas automaticamente pelo pipeline a partir dos Conventional Commits desde a última tag, publicadas junto à release no repositório.

O CHANGELOG é mantido em mobile/CHANGELOG.md, agrupado por Features e Bug Fixes, com links para commits e PRs. São praticados dois tipos de release: **interna** (cada merge publicado em Homologação, para validação) e **externa** (versão com tag publicada em Produção e APK disponível para instalação).

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

O estado dos ambientes é acompanhado pelo histórico do GitHub Actions (badges de CI/CD no README), pela aba Deployments de cada environment (versão implantada, data e responsável), pelos smoke tests automáticos pós-deploy e pelas notificações de falha enviadas pelo GitHub. Por se tratar de um site estático sem backend próprio, ferramentas como Prometheus/Grafana não são necessárias nesta fase.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.
Rework Rate	Proporção de implantações não planejadas feitas para corrigir problemas visíveis ao usuário.

Coleta e metas iniciais do projeto:

Métrica	Como é coletada	Meta inicial
Deployment Frequency	Nº de execuções do job deploy-producao (aba Deployments)	≥ 1 release por semana
Lead Time for Changes	Data do 1º commit do PR x data do deploy em produção	< 2 dias
Change Failure Rate	Releases seguidas de rollback ou hotfix / total de releases	< 15%
MTTR	Abertura da issue de defeito crítico x deploy da correção ou rollback	< 1 hora
Rework Rate	Deploys não planejados (hotfix/rollback) / total de deploys	< 10%

10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes, deploys) e os logs do sistema de versionamento constituem, por si, o principal insumo de auditoria de GCS, dispensando relatórios manuais na maior parte dos casos.

Auditorias planejadas:

- **Auditoria de configuração (a cada release):** verificar que tag, CHANGELOG, GitHub Release (APK e zip) e imagem no GHCR correspondem ao mesmo commit.
- **Auditoria de conformidade (mensal):** nenhum commit direto na main, todos os PRs com SM vinculada, aprovação e CI verde, e branches antigas removidas.
- **Verificação em laboratório:** apresentação deste PGCS e do repositório ao professor, conforme a ordem de publicação no Teams.

11. Marcos

Marcos internos do esforço de GCS/DevOps (datas previstas):

Marco	Descrição	Previsão
M1	Repositório criado com estrutura inicial, README, .gitignore e licença	22/09/2026
M2	Proteção da main configurada e canal de comunicação (Discussions) publicado	22/09/2026
M3	Pipeline de CI operacional: primeiro PR de feature integrado com CI verde	23/09/2026
M4	Baseline inicial v0.1.0 e publicação do PGCS (PDF) no Teams	23/09/2026
M5	Pipeline de CD: deploy automático em Homologação com smoke test	24/09/2026
M6	Primeira release automatizada (release-please) implantada em Produção	30/09/2026
M7	IaC aplicada: Terraform com estado remoto gerenciando repositório e environments	07/10/2026
M8	Integração do app com a API .NET do MoneyHub (feature flag)	11/2026
M9	Testes E2E automatizados em Homologação (integration_test)	11/2026

12. Treinamento e Recursos

Pessoal: 1 desenvolvedor (Leonardo Lamin) exercendo todos os papéis, 1 colega revisor de PRs e o professor da disciplina como stakeholder.

- **Ferramentas (custo zero):** GitHub Free com repositório público (Actions, Pages, GHCR, Environments com revisores), HCP Terraform Free, Flutter SDK, VS Code e Android Studio.

- **Treinamento:** conteúdo das aulas de GCS; documentação oficial do GitHub Actions, Flutter (testes) e Terraform; especificações Conventional Commits e SemVer.
- **Onboarding:** docs/onboarding.md (canal, acordos de trabalho e setup) e CONTRIBUTING.md (fluxo de mudança passo a passo).

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas e serviços de terceiros integrados ao produto também são itens de configuração e devem ser versionados explicitamente (arquivo de dependências travado — lockfile).

- Análise automática de vulnerabilidades em dependências (SCA) integrada ao pipeline de CI.
- Atualizações de dependências propostas automaticamente (ex.: Dependabot/Renovate) e submetidas ao mesmo fluxo de PR.

Dependências de terceiros atuais: Flutter SDK, flutter_lints (desenvolvimento), GitHub Actions referenciadas por versão (ex.: actions/checkout@v4) e a imagem base nginx:1.27-alpine. Não há subcontratados ou fornecedores externos. A API .NET do MoneyHub (repositório próprio) será consumida como dependência externa versionada: a versão do contrato utilizada fica registrada no README, e mudanças incompatíveis no contrato acionam o CCM.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Jobs pr-title, build-test (format, analyze, testes, cobertura, build web), security (gitleaks, Dependency Review) e docker-build.	Abertura/atualização de Pull Request
Merge	Squash merge da mudança na main; branch excluída.	Aprovação do PR + CI verde
CD — Staging	Build único (web, APK, imagem no GHCR), deploy em /staging e smoke test.	Merge na branch principal
Gate de Release	PR de release do release-please → tag vX.Y.Z + GitHub Release; aprovação no environment producao.	Merge do PR de release
CD — Produção	Deploy no GitHub Pages, APK/zip anexados à release, imagem X.Y.Z/latest, smoke test.	Aprovação do gate de release
Rollback	Reimplanta o artefato de uma release anterior, sem recompilar.	Acionamento manual (rollback.yml)

Nota: este plano é revisado a cada release ou mudança de processo; suas alterações seguem o mesmo fluxo de PR e são registradas no Histórico de Revisões, mantendo os quatro pilares da GCS — identificação, controle de versão, controle de mudanças e auditoria/status — suportados por automação.